Soluções em JavaScript — Relações Binárias

Estrutura Comum de Entrada

Para todos os exercícios, utilizaremos a biblioteca `readline-sync` para simplificar a leitura do teclado no Node.js. A função `readSet` abaixo será utilizada para capturar os conjuntos conforme as instruções.

1. Produto Cartesiano

```
const readline = require('readline-sync');

function readSet(name) {
    let set = [];
    console.log(`Digite os elementos do conjunto ${name} (
       negativo para parar):`);
    while (true) {
        let val = readline.questionInt('> ');
        if (val < 0) break;
        if (!set.includes(val)) set.push(val);
    }
    return set;
}

const A = readSet('A');
const B = readSet('B');

let product = [];
for (let a of A) {
    for (let b of B) {
        product.push(`(${a}, ${b})`);
    }
}

console.log(`A x B = {${product.join(', ')}}`);
```

2. União de Produtos Cartesianos

```
const readline = require('readline-sync');

function readSet(name) {
    let set = [];
    console.log(`Digite os elementos do conjunto ${name}:`);
```

```
    while (true) {
        let val = readline.questionInt('> ');
        if (val < 0) break;
        if (!set.includes(val)) set.push(val);
    }
    return set;
}

const A = readSet('A');
const B = readSet('B');

let union = new Set();

// A x B
for (let a of A) {
    for (let b of B) {
        union.add(`(${a}, ${b})`);
    }
}

// B x A
for (let b of B) {
    for (let a of A) {
        union.add(`(${b}, ${a})`);
    }
}

console.log(`(A x B) U (B x A) = {${Array.from(union).join(', ')
   }}`);
```

3. Propriedades de uma Relação

```
const readline = require('readline-sync');

function readSet(name) {
    let set = [];
    console.log(`Conjunto ${name} (negativo p/ fim):`);
    while (true) {
        let v = readline.questionInt('> ');
        if (v < 0) break;
        if (!set.includes(v)) set.push(v);
    }
    return set;
}

const A = readSet('A');
console.log('Informe os pares da rela  o R (ex: 1 2). Use -1
```

```
   para terminar.');
let R = [];
while (true) {
    let x = readline.questionInt('x: ');
    if (x < 0) break;
    let y = readline.questionInt('y: ');
    R.push([x, y]);
}

const hasPair = (x, y) => R.some(p => p[0] === x && p[1] === y);

// Reflexividade: para todo a em A, (a, a) em R
let reflexiva = A.every(a => hasPair(a, a));

// Simetria: para todo (x, y) em R, (y, x) em R
let simetrica = R.every(p => hasPair(p[1], p[0]));

// Antissimetria: se (x, y) e (y, x) em R, entao x = y
let antissimetrica = R.every(p => {
    if (hasPair(p[1], p[0])) return p[0] === p[1];
    return true;
});

// Assimetria: se (x, y) em R, entao (y, x) nao esta em R
let assimetrica = R.every(p => !hasPair(p[1], p[0]));

console.log(`Reflexiva: ${reflexiva ? 'Sim' : 'Nao'}`);
console.log(`Simetrica: ${simetrica ? 'Sim' : 'Nao'}`);
console.log(`Antissimetrica: ${antissimetrica ? 'Sim' : 'Nao'}`);
console.log(`Assimetrica: ${assimetrica ? 'Sim' : 'Nao'}`);
```

4. Matriz de Relação

```
const readline = require('readline-sync');

function readSet(name) {
    let set = [];
    console.log(`Conjunto ${name}:`);
    while (true) {
        let v = readline.questionInt('> ');
        if (v < 0) break;
        if (!set.includes(v)) set.push(v);
    }
    return set;
}

const A = readSet('A');
```

```
const B = readSet('B');
console.log('Informe os pares de R. -1 p/ fim.');
let R = [];
while (true) {
    let x = readline.questionInt('x: ');
    if (x < 0) break;
    let y = readline.questionInt('y: ');
    R.push([x, y]);
}

const hasPair = (x, y) => R.some(p => p[0] === x && p[1] === y);

console.log('Matriz de Relacao:');
for (let a of A) {
    let row = [];
    for (let b of B) {
        row.push(hasPair(a, b) ? 1 : 0);
    }
    console.log(`| ${row.join(' ')} |`);
}
```